```
template<typename T>
void threadsafe_queue<T>::push(T new_value)
{
     std::shared_ptr<T> new_data(
        std::make_shared<T>(std::move(new_value)));
     std::unique_ptr<node> p(new node);
     {
        std::lock_guard<std::mutex> tail_lock(tail_mutex);
        tail->data=new_data;
        node* const new_tail=p.get();
        tail->next=std::move(p);
        tail=new_tail;
     }
     data_cond.notify_one();
}
```

As already mentioned, the complexity is all in the *pop* side, which makes use of a series of helper functions to simplify matters. The next listing shows the implementation of `wait_and_pop()` and the associated helper functions.

Listing 6.9. A thread-safe queue with locking and waiting: `wait_and_pop()`

```
template<typename T>
class threadsafe_queue
{
private:
    node* get_tail()
    {
        std::lock_guard<std::mutex> tail_lock(tail_mutex);
        return tail;
    }

    std::unique_ptr<node> pop_head()     <-(1)
    {
        std::unique_ptr<node> old_head=std::move(head);
        head=std::move(old_head->next);
        return old_head;
    }

    std::unique_lock<std::mutex> wait_for_data()     <-(2)
    {
        std::unique_lock<std::mutex> head_lock(head_mutex);
        data_cond.wait(head_lock,[&]{return head.get()!=get_tail();});
        return std::move(head_lock);          <-(3)
    }
```